CST2550 Software Engineering Management and Development
TV Programme Scheduling System
Group Coursework - Project Management Report
Submission Date: 10th April 2026

Team Leader (Scrum Master)	Khrisvin	Nepaulsing	M01003462
Secretary	Veedoorsingh	Gopal	M01013795
Developer	Mehdi	Bhoyroo	M00983496
Developer	Navneet	Sookraz	M01000843
Tester	Pranay	Poheith	M01013276

Contents

Project Description 1

Team Roles and Responsibilities 1

Tools and Collaboration Platforms 2

Meeting Management and Communication 2

Workload Division 3

Project Planning 3

- Planning Approach 3
- Work Breakdown Structure (WBS) 4
- Work Timeline 4
- Gantt Chart 4

Project Design and Implementation Management 5

- Activity Diagram 5
- Flowchart 5
- Use Case 5

Testing Management 5

Risk Management 6

Conclusion 6

Appendices 7

- Meeting Minutes 7
- Sprint Plans 7
- Stand-Up Records 7
- Retrospectives 7

Project Description

The TV Programme Scheduling System was developed to help a scheduling user organise television programmes more efficiently. The main aim of the project was to create a desktop application that allows a user to add schedules, remove schedules, view schedules by channel and receive support when searching for the next available free slot.

From a management perspective, the project had two linked objectives. The first was to deliver a working software product that met the functional requirements of the module. The second was to manage the work in a structured way so that planning, communication, testing and final integration were completed on time.

The scope of the project included requirements discussion, interface design, implementation in Visual Studio, version control through GitHub, diagram preparation in draw.io, testing, and report preparation. The team therefore had to balance technical work with coordination and documentation throughout the development period.

Team Roles and Responsibilities

A five-member team was formed for the project. Roles were allocated so that each member had a clear area of responsibility while still being able to support the wider development process. This helped reduce confusion, improved accountability, and made progress easier to monitor.

Team Member	Role	Responsibilities
Khrisvin Nepaulsing	Team Leader (Scrum Master)	Coordinated the project, organised meetings, tracked task progress, followed up on deadlines and supported final integration.
Veedoorsingh Gopal	Secretary	Recorded meeting notes, documented decisions, helped maintain communication records and supported report organisation.
Mehdi Bhoyroo	Developer	Worked on implementation tasks, supported feature development and helped with code integration and issue fixing.
Navneet Sookraz	Developer	Worked on implementation tasks, supported interface and logic development, and contributed code through GitHub.
Pranay Poheith	Tester	Prepared and ran tests, reported defects, checked outputs against expected behaviour and supported verification before submission.

The chosen roles reflected the needs of the project. A Scrum Master role was useful because the team needed one member to keep work moving and make sure responsibilities were clear. The Secretary role supported documentation and decision tracking, which was important for the project management report. The two developers focused mainly on implementation tasks, while the tester concentrated on checking whether the system behaved correctly. Even with

defined roles, members still communicated regularly and supported one another when clarification or integration work was needed.

Tools and Collaboration Platforms

The team used a small set of practical tools during the project. Each tool was selected because it directly supported either development, communication, documentation or coordination.

Tool	Purpose	Why it was useful
Visual Studio	Main development environment	Allowed the team to write, run and debug the system in one place.
GitHub	Version control and code sharing	Made it easier to manage updates, keep a shared code base and monitor contributions.
Windows Prompt	Command-line support	Useful for running and checking certain project files and commands during development.
draw.io	Diagrams and visual planning	Used to prepare system diagrams and other report visuals in a clear format.
WhatsApp	Communication and quick updates through group calls and messages	Supported rapid discussion when the team was not in the same location.

GitHub was especially important because it provided a shared repository and helped the team manage changes more safely. Visual Studio supported the practical development side, while draw.io was useful when the group needed diagrams for both the technical and project management reports. WhatsApp was not a development tool, but it was central to coordination because it allowed the team to organise calls and keep quick communication active outside formal meetings.

Meeting Management and Communication

Communication was handled through a combination of face-to-face meetings and WhatsApp group calls. The team held meetings both at home and at the university, depending on availability. This flexible approach was practical because it allowed the members to continue discussing the project even when they were not all in the same place.

Face-to-face meetings were mainly used for planning, reviewing progress, discussing technical issues and dividing work. WhatsApp calls were used for follow-up discussions, short updates and urgent clarifications. This helped the team maintain regular contact without relying only on one communication method.

Communication method	How it was used	Benefit
Meetings at university	Planning, reviewing progress, checking assigned tasks	Useful for direct discussion and faster group decisions.

Meetings at home	Longer working sessions, integration and report discussion	Allowed focused collaboration in a practical setting.
WhatsApp group calls/messages	Quick updates, clarifications and scheduling discussions	Kept communication active between meetings.

Meeting notes were mainly supported by the secretary, while key decisions were also reinforced through group discussion. This created a simple but effective communication structure. Although the team did not depend on complex project management software, the combination of meetings, calls and role allocation helped maintain organised collaboration.

Workload Division

Work was divided according to role and practical strengths. This was important because the project involved both technical tasks and coordination tasks. Assigning responsibilities clearly reduced duplication of effort and made progress easier to measure.

Area of work	Person(s) responsible	Examples of tasks
Project coordination	Khrisvin	Monitoring progress, organising meetings, and checking completion of planned tasks.
Documentation and meeting records	Veedoorsingh	Recording notes, helping organise report material and decisions.
Development	Mehdi and Navneet	Implementing features, editing code, supporting integration.
Testing and verification	Pranay	Preparing test cases, checking outputs, reporting defects and confirmations.
Shared report preparation	All members with coordination support	Reviewing content, diagrams and consistency before submission.

The workload was therefore divided in a responsibility-based way rather than randomly. This made the project more manageable because each person knew what was expected. At the same time, members still discussed issues together during meetings, which helped when tasks overlapped or when one area affected another, such as implementation and testing.

Project Planning

The team used a simple Agile-style approach. Instead of trying to complete everything at once, the project was broken into smaller stages so that planning, implementation, checking and improvement could happen gradually. This was suitable for the group because it made the work easier to organise and allowed problems to be identified earlier.

Planning Approach

The planning approach was practical rather than highly formal. The team first identified the core features required in the system and then linked them to responsibilities. This allowed the

group to focus on completing essential work before moving to improvements and supporting materials. The Scrum Master monitored whether the work remained aligned with deadlines and whether tasks needed to be redistributed.

Work Breakdown Structure (WBS)

The work was decomposed into manageable categories before full implementation. The main work packages included planning and coordination, software design, implementation, testing, report writing and final review. Breaking the project into these areas helped the team sequence tasks logically and avoid confusion about what should be done first.

Work package	Typical activities
Planning and coordination	Role allocation, meetings, task tracking, decision making.
Software design	Discussing structure, interface flow and required diagrams.
Implementation	Coding main features and integrating project components.
Testing	Running checks, identifying errors and confirming expected outputs.
Documentation	Preparing technical report, project management report and diagrams.
Final review	Checking consistency, organising files and preparing for submission.

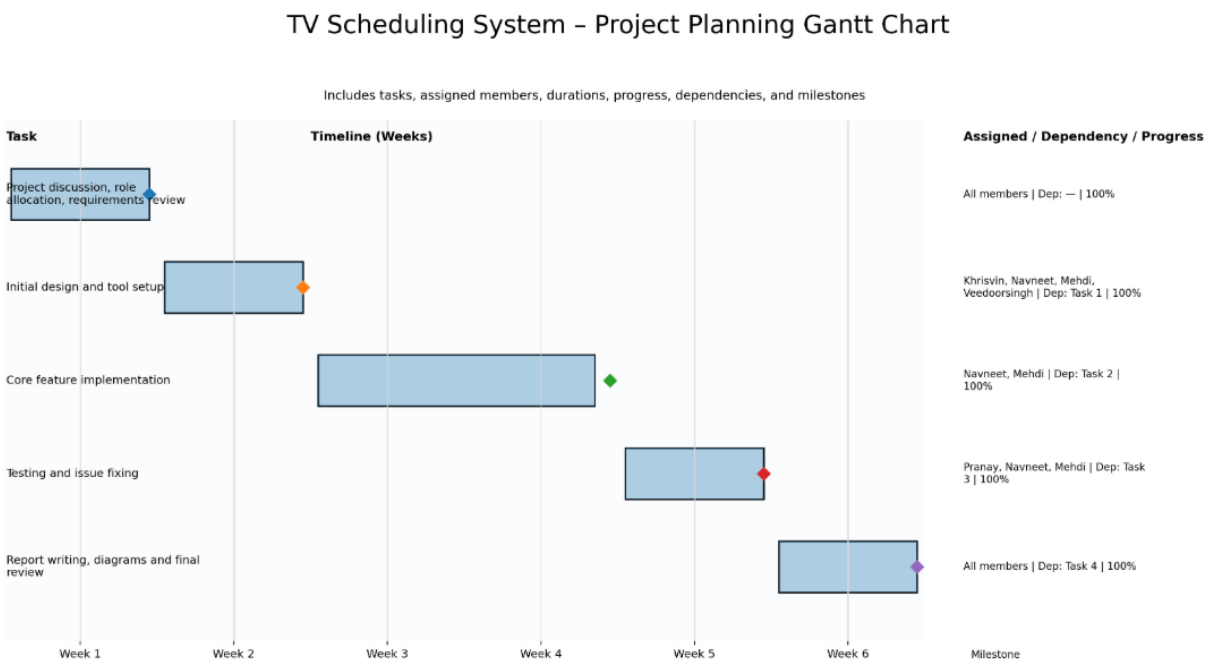
Work Timeline

The timeline was used as a guide for the order in which work should be completed. Because university projects often involve timetable constraints and changing availability, the timeline needed some flexibility. However, the general sequence remained stable and supported orderly progress from planning to submission.

Stage	Main activities	Responsible members	Expected outcome
Week 1	Project discussion, role allocation, requirement review	All members	Shared understanding of project scope.
Week 2	Initial design decisions and setup of tools	Scrum Master, developers, secretary	Clear working structure and repository use.
Weeks 3-4	Implementation of core scheduling features	Developers with team support	Main functions developed.
Week 5	Testing and issue fixing	Tester with developers	Errors identified and corrected.
Week 6	Report writing, diagrams and final review	All members	Submission materials prepared.

Although the timeline was simplified, it helped the team understand what needed attention at each stage. When minor delays occurred, communication through meetings and WhatsApp helped the team adjust without losing the overall direction of the project.

Gantt Chart



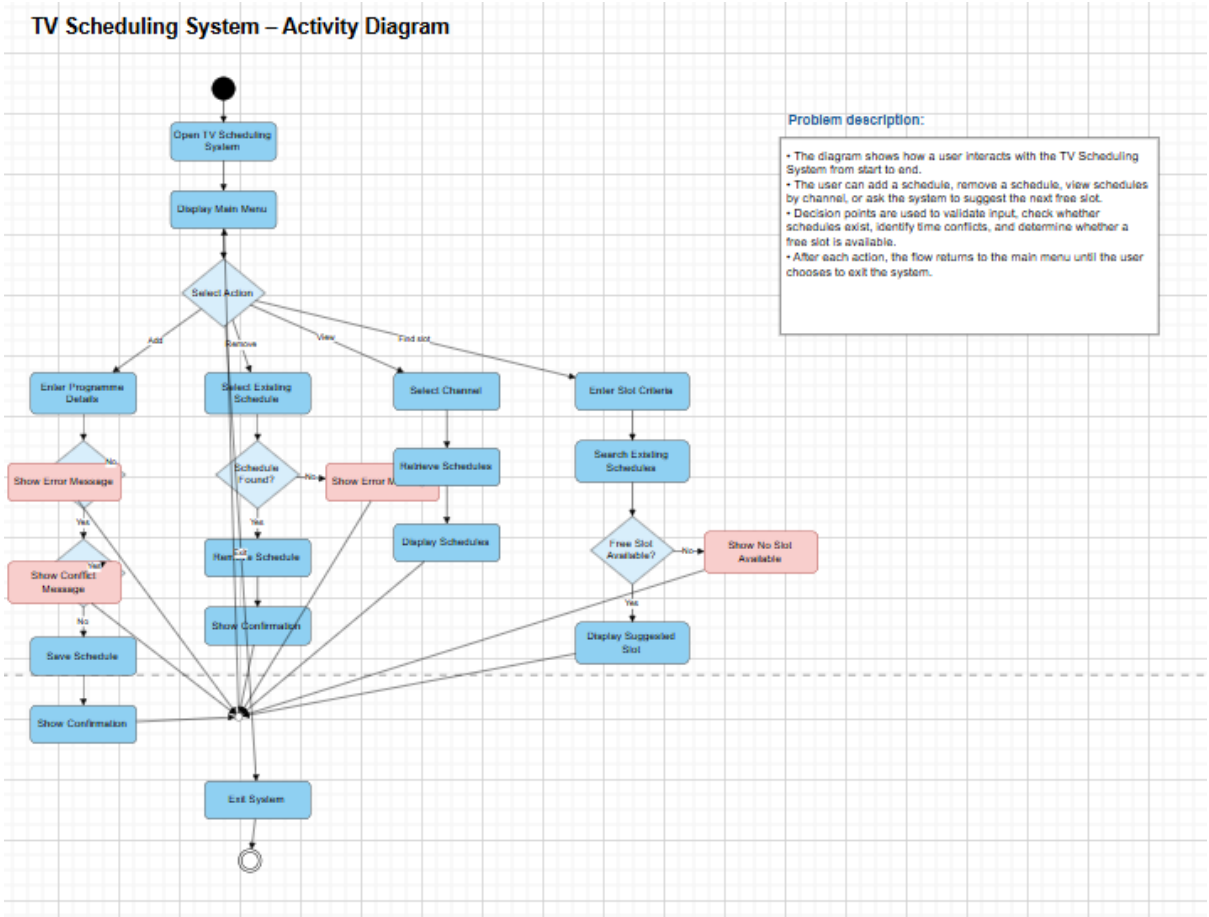
Project Design and Implementation Management

Managing implementation required continuous coordination between planning, coding and review. The team needed to make sure that design decisions were understood before or during implementation so that the software remained consistent.

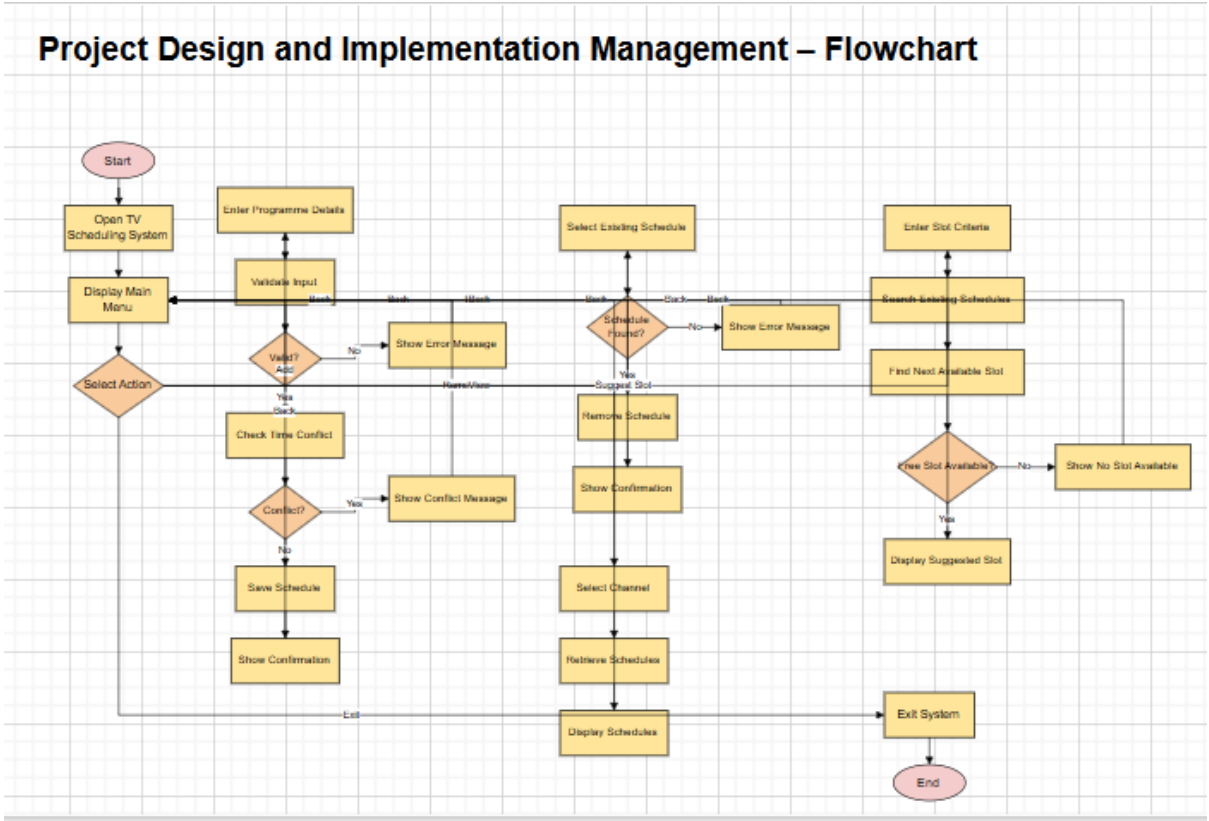
Visual representations created in draw.io supported this process by helping members discuss the system more clearly. Diagrams were useful because they reduced ambiguity and gave the team a common point of reference during both report writing and feature discussion.

Implementation tasks were mainly handled by the developers, but progress was tracked through team discussions. The Scrum Master checked whether assigned tasks were advancing, while the secretary helped ensure that decisions were not lost between meetings. This coordination process reduced the risk of members working in isolation and made final integration more manageable.

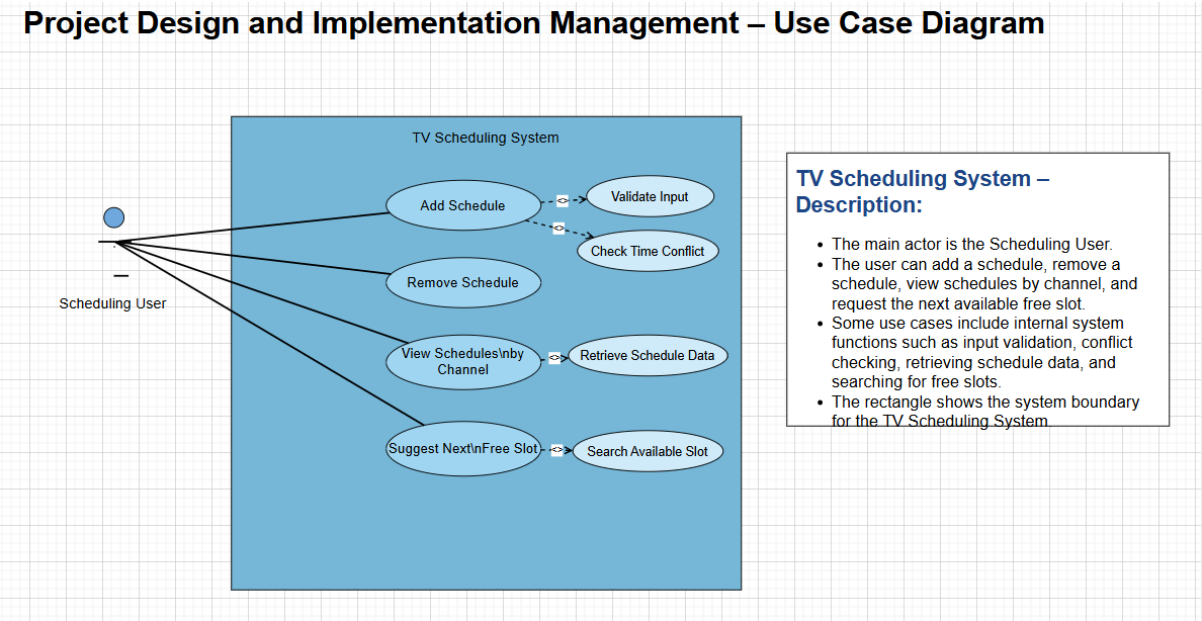
Activity Diagram


TV Scheduling System – Activity Diagram
Open TV Scheduling System
Display Main Menu
Select Action
Add
Remove
View
Find slot
Enter Programme Details
Select Existing Schedule
Select Channel
Enter Slot Criteria
Show Error Message
No
Yes
Schedule Found?
No
Show Error M
Yes
Retrieve Schedules
Search Existing Schedules
Show Conflict Message
Yes
No
Remove Schedule
Exit
Display Schedules
Free Slot Available?
No
Show No Slot Available
Yes
Save Schedule
Show Confirmation
Display Suggested Slot
Show Confirmation
Exit System
Problem description:
• The diagram shows how a user interacts with the TV Scheduling System from start to end.
• The user can add a schedule, remove a schedule, view schedules by channel, or ask the system to suggest the next free slot.
• Decision points are used to validate input, check whether schedules exist, identify time conflicts, and determine whether a free slot is available.
• After each action, the flow returns to the main menu until the user chooses to exit the system.

Flowchart



Use Case



Testing Management

Testing management was mainly led by the team tester, Pranay Poheith, with support from the developers when issues were found. The purpose of testing was to confirm that the system behaved as expected and that the main scheduling actions could be completed correctly.

The main areas checked included adding schedules, removing schedules, viewing schedules and checking availability for the next free slot. Valid cases were important, but the team also needed to observe invalid or problematic cases, such as conflicts, missing information or unexpected outputs.

Test area	Example check	Reason for test	Outcome
Add schedule	Check that a valid schedule can be saved	Confirms main core feature works	Verified during testing stage.
Remove schedule	Check that a selected schedule can be removed	Confirms existing data can be updated	Verified during testing stage.
View schedules by channel	Check that schedules display correctly	Ensures the user can read stored information	Verified during testing stage.
Suggest next free slot	Check that the system suggests an available time	Supports decision making and conflict avoidance	Verified during testing stage.

When problems were identified, the tester communicated them back to the team so that corrections could be made. This feedback loop was important because it linked testing directly to improvement rather than treating testing as a separate final step only.

Risk Management

Risk management was necessary because software projects can face both technical and coordination problems. The team, therefore, needed to think not only about coding risks but also about communication and deadline risks.

Risk	Potential impact	Mitigation approach
Communication gaps	Tasks may be delayed or duplicated	Regular meetings and WhatsApp calls kept members updated.
Integration issues	Different parts may not work together smoothly	Use of GitHub and team review supported controlled integration.
Testing delays	Errors may remain close to submission	A dedicated tester role helped keep verification visible.
Deadline pressure	Quality may fall if work is left too late	Task division and follow-up by the Scrum Master supported earlier completion.
Documentation gaps	Important decisions may be forgotten	Secretary role helped capture notes and support report preparation.

These risks were managed with simple but practical methods. The team did not use a formal enterprise risk framework, but regular communication, clear roles and shared tools reduced uncertainty and helped problems to be addressed quickly.

Conclusion

Overall, the project was managed through clear role allocation, practical communication and a simple Agile-style planning approach. The team used face-to-face meetings, WhatsApp communication, Visual Studio, GitHub, Windows Prompt and draw.io to support development and coordination. This combination of tools and roles helped the group produce a structured workflow around the TV Programme Scheduling System.

The project management approach was effective because responsibilities were defined, progress was monitored and testing was given a clear place in the process. While the management structure remained simple, it was appropriate for the scale of the project and supported completion of the main objectives within the module context.

Appendices

Appendix A: Meeting Minutes

Weekly meeting records showing attendance, agenda items, discussion points, decisions taken, and action items assigned to each team member.

Appendix B: Sprint Plans

Weekly sprint planning records outlining sprint goals, planned tasks, task allocation, and expected deliverables for the TV Schedule System project.

Appendix C: Stand-Up Records

Weekly stand-up records summarizing what each member completed, what they planned to do next, and any blockers or issues affecting progress.

Appendix D: Retrospectives

Weekly retrospective records identifying what went well, what did not go well, and the improvements agreed for the following sprint.